

Diretrizes para implementação do projeto no OMNeT++ e INET

1. Objetivo

Implementar e avaliar uma rede de comunicação entre drones e equipes de resgate, na qual o Algoritmo do Morcego determina o reposicionamento de drones quando obstáculos físicos comprometem a entrega de alertas de vítimas.

Pergunta de pesquisa

Em cenários de busca e resgate com drones, equipes móveis e obstáculos físicos, uma estratégia de reposicionamento baseada no Algoritmo do Morcego altera a entrega de alertas em comparação com a operação sem reposicionamento?

A avaliação deve considerar a política completa:

1. Identificação de falhas na comunicação.
2. Verificação da presença de obstáculo.
3. Definição de uma nova posição pelo algoritmo.
4. Deslocamento do drone.
5. Nova tentativa de transmissão.

O trabalho não pretende demonstrar que o algoritmo sempre encontra a melhor posição ou que cada perda foi causada exclusivamente por um obstáculo.

2. Escopo do projeto

O projeto deve representar:

- Drones móveis operando em uma rede aérea ad hoc.
- Comunicação entre drones e equipes de resgate.
- Equipes móveis no solo.
- Vítimas em posições fixas.
- Obstáculos físicos estáticos.
- Comunicação sem fio IEEE 802.11.

- Encaminhamento de mensagens pelo AODV padrão do INET.
- Alertas periódicos sobre vítimas.
- Confirmações de recebimento.
- Retransmissões na camada de aplicação.
- Identificação de falhas por ausência de confirmação.
- Verificação geométrica simplificada de obstáculos.
- Reposicionamento de drones pelo Algoritmo do Morcego.
- Coleta estruturada de métricas.

Não fazem parte do escopo:

- Processamento de imagens para identificar vítimas.
- Implementação real de câmera, LiDAR ou visão computacional.
- Controle de voo de baixo nível.
- Alteração do funcionamento interno do AODV.
- Desenvolvimento de um novo protocolo de roteamento.
- Obstáculos móveis.
- Modelagem detalhada de bateria.
- Tomada de decisão humana sobre o atendimento.
- Simulação clínica ou operacional do resgate.

A autonomia representa a capacidade de o drone executar automaticamente decisões de comunicação e reposicionamento dentro das regras implementadas.

3. Configuração geral do cenário

Parâmetro	Definição inicial
Simulador	OMNeT++
Framework	INET
Área	1000 × 1000 m
Duração	900 s
Quantidade inicial de drones	4
Quantidade de vítimas	1 ou 2
Quantidade de equipes	1, 5, 10 ou 15

Parâmetro	Definição inicial
Quantidade de obstáculos	2
Altitude máxima dos drones	20 m
Velocidade inicial dos drones	13 m/s
Mobilidade dos drones	Gauss–Markov tridimensional
Mobilidade das equipes	Random Walk
Mobilidade das vítimas	Estática
Tecnologia sem fio	IEEE 802.11
Taxa física inicial	1 Mbit/s
Transporte	UDP
Roteamento	AODV padrão do INET
Repetições por configuração	Pelo menos 30
Tempo operacional considerado	15 minutos

Os parâmetros definitivos devem estar centralizados nos arquivos `.ini`.

Os valores utilizados na dissertação, nos experimentos e nos resultados devem corresponder exatamente aos valores configurados.

4. Organização da rede

A rede deve possuir os seguintes elementos:

- Drones.
- Equipes de resgate.
- Vítimas.
- Obstáculos.
- Gerenciador de cenário.
- Módulo de coleta de métricas.

Os drones devem formar uma rede aérea ad hoc.

Cada drone pode:

- Transmitir seus próprios alertas.
- Receber atualizações das equipes.
- Encaminhar pacotes de outros drones utilizando o AODV.
- Identificar falhas de comunicação.

- Executar reposicionamento quando necessário.

As equipes:

- Informam sua posição periodicamente.
- Recebem alertas de vítimas.
- Respondem com confirmações.

As vítimas:

- Permanecem em posições fixas.
- São associadas a drones responsáveis.
- Geram eventos abstratos de detecção.

Não é necessário implementar a identificação visual real da vítima.

5. Papel do AODV

O AODV deve ser utilizado como protocolo de roteamento disponível no INET, sem modificações.

Ele deve permitir:

- Descoberta de rotas entre drones e equipes.
- Encaminhamento de mensagens por drones intermediários.
- Atualização de rotas conforme a mobilidade dos nós.

Um alerta pode seguir diretamente:

Drone → Equipe

Ou utilizar drones intermediários:

Drone emissor → Drone intermediario → Equipe

A confirmação também pode retornar por uma rota disponível:

Equipe → Drone intermediario → Drone emissor

O projeto não deve limitar os alertas a:

```
applicationIpTtl=1
```

quando o objetivo for avaliar uma FANET com encaminhamento entre drones.
O valor de TTL deve permitir múltiplos saltos.

6. Componentes da implementação

6.1. Gerenciador de cenário

O gerenciador deve:

- Posicionar vítimas.
- Agendar os instantes de detecção.
- Identificar os drones responsáveis.
- Controlar os cenários com uma ou mais vítimas.
- Garantir que os mesmos parâmetros sejam utilizados nas execuções comparadas.

Nome sugerido:

```
SarScenarioManager
```

Nos cenários principais:

- Uma vítima corresponde a um drone responsável.
- Duas vítimas correspondem a dois drones responsáveis.

A associação pode ser configurada diretamente para garantir reprodutibilidade.

Caso exista seleção automática, deve ser utilizado o drone elegível mais próximo da vítima no instante de detecção.

6.2. Representação da vítima

Cada vítima deve possuir:

```
victimId  
position  
detectionTime  
assignedDroneIndex  
active
```

Nome sugerido:

StaticVictim

A vítima permanece ativa até o encerramento da simulação ou até uma condição operacional configurada.

A confirmação de um alerta não significa automaticamente que a vítima já foi atendida.

Uma vítima pode ser observada pelo mesmo drone ou outro drone após x segundos. Fazendo o encaminhamento da mensagem de alerta novamente para equipes.

6.3. Aplicação da equipe

A aplicação da equipe deve:

- Enviar atualizações periódicas de posição.
- Receber alertas.
- Registrar entregas.
- Identificar mensagens duplicadas.
- Enviar confirmações.

Nome sugerido:

TeamApp

6.4. Aplicação do drone

A aplicação do drone deve:

- Receber atualizações das equipes.
- Manter informações recentes das equipes conhecidas.
- Gerar alertas de vítimas.
- Enviar alertas por UDP.
- Solicitar encaminhamento pela pilha de rede.
- Controlar retransmissões.
- Validar confirmações.
- Identificar falhas.

- Consultar o sensor abstrato.
- Solicitar reposicionamento.
- Registrar eventos e métricas.

Nome sugerido:

DroneApp

6.5. Sensor abstrato

O sensor deve:

- Receber a posição atual do drone.
- Receber a última posição conhecida da equipe.
- Verificar a existência de um obstáculo observável na direção do enlace.
- Retornar a posição estimada do obstáculo quando houver detecção.

Nome sugerido:

AbstractObstacleSensor

O sensor não deve processar imagens reais.

Seu comportamento representa uma abstração de um sensor embarcado.

6.6. Algoritmo do Morcego

O algoritmo deve:

- Receber a posição atual do drone.
- Receber a posição conhecida da equipe.
- Receber a posição estimada do obstáculo.
- Gerar posições candidatas.
- Avaliar a função de aptidão.
- Retornar uma posição viável.

Nome sugerido:

BatAlgorithm

6.7. Avaliação de posições

A avaliação deve considerar:

- Proximidade da equipe.
- Presença de obstáculo.
- Distância de deslocamento.
- Restrições da área.
- Limites de altitude.
- Preservação da conectividade com outros drones, quando aplicável.

Nome sugerido:

RepositionFitness

6.8. Mobilidade com reposicionamento

O módulo de mobilidade deve:

- Executar a mobilidade Gauss–Markov.
- Interromper temporariamente a trajetória normal quando houver reposicionamento.
- Deslocar o drone gradualmente.
- Informar a conclusão do movimento.
- Retomar a mobilidade normal.

Nome sugerido:

BaGaussMarkovMobility

O drone não deve ser teletransportado.

6.9. Registro de métricas

O módulo de métricas deve:

- Registrar alertas gerados.
- Registrar tentativas transmitidas.
- Registrar entregas.

- Registrar confirmações.
- Registrar retransmissões.
- Registrar atrasos.
- Registrar ativações do BA.
- Registrar reposicionamentos.
- Registrar encaminhamentos por drones intermediários.

Nome sugerido:

ExperimentMetrics

7. Tipos de mensagens

7.1. Atualização da equipe

Nome sugerido:

TeamUpdate

Campos mínimos:

```
teamId
messageId
sequenceNumber
position
timestamp
```

Periodicidade inicial:

$$T_{\text{team}} = 1 \text{ s}$$

A equipe transmite sua posição por broadcast.

Cada drone registra apenas informações recebidas efetivamente pela rede.

Não utilizar acesso direto à posição interna da equipe como substituto das mensagens recebidas.

7.2. Atualização do drone

Nome sugerido:

DroneStatus

Campos mínimos:

```
droneId  
messageId  
sequenceNumber  
position  
timestamp
```

Finalidades:

- Informar a presença do drone.
- Permitir observação dos drones vizinhos.
- Auxiliar na avaliação da conectividade durante o reposicionamento.

Periodicidade sugerida:

$$T_{\text{drone}} = 2 \text{ s}$$

Esse valor deve ser configurável.

7.3. Alerta de vítima

Nome sugerido:

VictimAlert

Campos mínimos:

```
alertId  
messageId  
victimId  
sourceDroneId  
targetTeamId  
victimPosition  
creationTime  
attemptNumber
```

O alerta representa uma atualização sobre uma vítima ativa.

Periodicidade inicial:

$$T_{\text{alerta}} = 30 \text{ s}$$

Cada novo alerta periódico recebe um novo:

```
alertId
```

As retransmissões do mesmo alerta preservam:

```
alertId
```

e recebem um novo:

```
messageId
```

Não criar alertas concorrentes para a mesma vítima enquanto um alerta anterior ainda estiver pendente, salvo se essa concorrência for explicitamente implementada e justificada.

7.4. Confirmação

Nome sugerido:

```
VictimAck
```

Campos mínimos:

```
alertId
messageId
victimId
sourceDroneId
teamId
ackTimestamp
```

A equipe deve responder após receber um alerta.

O drone deve validar a correspondência entre a confirmação e a tentativa que originou a mensagem.

8. Descoberta e seleção das equipes

Cada drone deve manter uma tabela local com:

```
teamId  
networkAddress  
lastKnownPosition  
lastUpdateTime  
sequenceNumber
```

Regras:

1. Apenas atualizações recebidas pela rede podem inserir ou atualizar equipes.
2. Atualizações duplicadas não devem renovar registros.
3. Atualizações fora de ordem devem ser ignoradas.
4. Registros antigos devem expirar.
5. O endereço da equipe deve ser obtido do pacote recebido.
6. A equipe selecionada deve ser a equipe conhecida mais próxima do drone.
7. O AODV deve ser responsável por descobrir uma rota até a equipe escolhida.

A ausência de uma equipe conhecida deve ser registrada como falha operacional.

Não utilizar um gerenciador global para entregar ao drone a posição atual de equipes que ele não conhece.

9. Detecção da vítima

A identificação da vítima deve ser modelada como um evento abstrato.

Não implementar:

- Reconhecimento de imagens.
- Rastreamento visual.
- Segmentação de imagem.
- Processamento de vídeo.

Ao ocorrer a detecção:

1. O gerenciador informa o drone responsável.
2. A vítima passa a ser considerada ativa.
3. O drone inicia o envio periódico de alertas.

Exemplo:

```
160 s: vítima 0 identificada.  
Drone responsável: drone 2.
```

No cenário com duas vítimas:

```
160 s: vítima 0 identificada pelo drone 2.  
260 s: vítima 1 identificada pelo drone 1.
```

Esses valores devem ser configuráveis.

10. Obstáculos

Os obstáculos devem ser:

- Estáticos.
- Posicionados dentro da área.
- Modelados com dimensões explícitas.
- Integrados ao modelo de propagação.

Dimensões iniciais definidas na reunião:

$$h = 5 \text{ m}$$

$$l = 3 \text{ m}$$

$$p = 2 \text{ m}$$

Em que:

- h : altura.
- l : largura.
- p : profundidade.

Essas dimensões devem ser utilizadas como ponto de partida.

Se forem substituídas por edifícios maiores, a alteração precisa ser justificada e refletida em todos os documentos.

A atenuação deve ocorrer somente quando o percurso de propagação efetivamente atravessar o obstáculo, conforme o modelo disponível no INET.

Modelo sugerido:

Os materiais e parâmetros elétricos devem ser declarados na configuração.

Não classificar esses obstáculos como dinâmicos.

O cenário é dinâmico devido à mobilidade dos drones e das equipes.

11. Coerência geométrica dos obstáculos

A presença de um obstáculo no cenário não garante obstrução do enlace.

Para cada situação avaliada, deve ser possível verificar:

1. Posição do drone.
2. Altitude do drone.
3. Posição da equipe.
4. Posição do obstáculo.
5. Altura do obstáculo.
6. Interseção entre o enlace e o obstáculo.

Exemplo:

Se o drone está a 12 m de altura e um obstáculo de 5 m está exatamente no meio do caminho até a equipe, a linha de visada pode passar acima do obstáculo.

Portanto, os obstáculos devem ser posicionados de forma coerente com a geometria do cenário.

Antes dos experimentos principais, executar cenários controlados para verificar:

- Enlace sem obstáculo.
- Enlace com obstáculo sem interseção.
- Enlace com obstáculo interceptando a trajetória do sinal.
- Efeito do reposicionamento sobre a linha de visada.

10. Obstáculos

Os obstáculos devem ser:

- Estáticos.
- Posicionados dentro da área de simulação.

- Modelados com dimensões explícitas.
- Integrados ao modelo de propagação.
- Distribuídos de modo a interferir, em determinadas situações, na comunicação entre drones e equipes.

Para aumentar a probabilidade de obstrução do enlace, recomenda-se representar obstáculos compatíveis com edificações presentes em áreas urbanas.

Dimensões iniciais sugeridas:

$$[h = 15, \text{m}][l = 20, \text{m}][p = 10, \text{m}]$$

Em que:

- (h): altura do obstáculo.
- (l): largura do obstáculo.
- (p): profundidade do obstáculo.

Considerando que os drones operam em altitudes de até $(20, \text{m})$, um obstáculo com $(15, \text{m})$ de altura pode interferir no enlace entre o drone e uma equipe no solo, dependendo da posição dos elementos e da trajetória do sinal.

Essa dimensão também preserva espaço para que o drone busque uma posição alternativa acima ou ao redor do obstáculo.

Os obstáculos devem representar estruturas fixas, como edificações, paredes ou blocos de concreto.

A atenuação do sinal deve ocorrer quando o percurso de propagação entre transmissor e receptor atravessar o obstáculo, conforme o modelo configurado no INET.

Modelo sugerido:

```
DielectricObstacleLoss
```

Os seguintes parâmetros devem ser definidos na configuração:

- Posição de cada obstáculo.
- Altura, largura e profundidade.
- Material utilizado.

- Propriedades elétricas do material.
- Distância mínima de segurança para o reposicionamento.

Antes dos experimentos principais, deve-se verificar:

1. Comunicação sem obstáculo entre drone e equipe.
2. Comunicação com obstáculo fora da linha de propagação.
3. Comunicação com obstáculo interceptando o enlace.
4. Efeito do reposicionamento sobre a linha de visada e a entrega dos alertas.

As dimensões podem ser ajustadas após testes preliminares, desde que as alterações sejam justificadas e mantidas consistentes entre os cenários com e sem reposicionamento.

Os obstáculos não devem ser classificados como dinâmicos. A variação do enlace decorre da mobilidade dos drones e das equipes em relação às estruturas fixas.

12. Identificação de falhas na comunicação

O mecanismo de decisão deve ser simples.

A condição inicial de falha é baseada na ausência de confirmação:

$$N_{\text{tentativas sem ACK}} \geq N_{\text{limiar}}$$

Em que:

- $N_{\text{tentativas sem ACK}}$ representa a quantidade de tentativas consecutivas sem confirmação.
- N_{limiar} representa o limite configurado.

Exemplo inicial:

$$N_{\text{limiar}} = 2$$

Quando o limite é atingido:

1. O drone verifica se a equipe continua conhecida.
2. O drone consulta o sensor abstrato.
3. O sensor verifica a presença de obstáculo.
4. Caso exista obstáculo, o reposicionamento pode ser solicitado.

Não utilizar RSSI ou PDR em janela temporal como condições obrigatórias se esses mecanismos não estiverem implementados.

Essas métricas podem ser registradas posteriormente como diagnóstico adicional.

A ausência de confirmação não prova que o obstáculo causou a falha.

A falha também pode resultar de:

- Perda do alerta.
- Perda do ACK.
- Ausência de rota.
- Mobilidade.
- Interferência.
- Concorrência pelo canal.

13. Parâmetros de retransmissão

Os parâmetros devem ser configuráveis no `.ini`.

Valores iniciais sugeridos:

Parâmetro	Valor inicial
Intervalo de atualização da equipe	1 s
Intervalo de atualização do drone	2 s
Intervalo entre novos alertas	10 s
Tempo de espera por ACK	2 s
Intervalo entre retransmissões	3 s
Máximo de tentativas por alerta	4
Limite para avaliação do reposicionamento	2 tentativas sem ACK
Validade máxima do alerta	25 s
Período de manutenção interna	0,1 s
Validade inicial do registro de equipe	30 s

Esses valores são iniciais e devem ser calibrados.

O tempo de espera por ACK deve ser compatível com:

- Descoberta de rota pelo AODV.
- Quantidade de saltos.
- Acesso ao meio.

- Mobilidade.
- Atrasos da pilha IEEE 802.11.

Deve valer:

$$N_{\text{limiar}} < N_{\text{maximo de tentativas}}$$

Essa condição reserva tentativas para depois do reposicionamento.

A validade do alerta deve ser inferior ao intervalo entre alertas periódicos quando houver apenas um alerta pendente por vítima.

14. Funcionamento do sensor abstrato

O sensor deve receber:

$$p_d = (x_d, y_d, z_d)$$

e:

$$p_e = (x_e, y_e, z_e)$$

Em que:

- p_d : posição atual do drone.
- p_e : última posição conhecida da equipe.

O sensor verifica se existe uma superfície observável na direção entre esses pontos.

Quando houver detecção:

$$p_o = (x_o, y_o, z_o)$$

Em que p_o representa a posição estimada do obstáculo.

O retorno deve indicar:

```
detected
```

ou:

```
notDetected
```

O alcance sensorial deve ser configurável:

```
obstacleSensorRange
```

Caso seja utilizado um alcance de 30 m, os cenários precisam garantir que os obstáculos relevantes possam ser observados nessa distância.

Não utilizar conhecimento global de obstáculos como se fosse informação efetivamente disponível ao drone.

Se a avaliação utilizar a geometria completa do cenário para testar posições candidatas, essa hipótese deve ser declarada como:

Conhecimento geométrico idealizado disponibilizado ao mecanismo de reposicionamento.

15. Condições para executar o Algoritmo do Morcego

O BA deve ser acionado somente quando:

1. Existe um alerta pendente.
2. O limite de tentativas sem ACK foi atingido.
3. A equipe continua conhecida.
4. O drone não está realizando outro reposicionamento.
5. Um obstáculo observável foi identificado.
6. A política está habilitada.

Parâmetro sugerido:

```
*,drone[*].app[0].baEnabled=true
```

No cenário de controle:

```
*,drone[*].app[0].baEnabled=false
```

Quando a política estiver desligada:

- O drone continua retransmitindo normalmente.
- A condição de falha pode ser registrada.

- A detecção do obstáculo pode ser registrada.
- Nenhum deslocamento especial deve ocorrer.

16. Função de aptidão

A função de aptidão deve considerar três critérios:

1. Aproximação da equipe.
2. Afastamento do obstáculo.
3. Distância necessária para o deslocamento.

Formulação:

$$J(x) = w_{\text{equipe}} C_{\text{equipe}}(x) + w_{\text{obstaculo}} C_{\text{obstaculo}}(x) + w_{\text{deslocamento}} C_{\text{deslocamento}}(x)$$

Os pesos devem satisfazer:

$$w_{\text{equipe}} + w_{\text{obstaculo}} + w_{\text{deslocamento}} = 1$$

Valores iniciais:

$$w_{\text{equipe}} = 0,60$$

$$w_{\text{obstaculo}} = 0,25$$

$$w_{\text{deslocamento}} = 0,15$$

16.1. Distância até a equipe

$$C_{\text{equipe}}(x) = \min \left(\frac{\|x - p_e\|}{d_{\text{referencia}}}, 1 \right)$$

Em que:

- x : posição candidata.
- p_e : posição conhecida da equipe.
- $d_{\text{referencia}}$: distância de normalização.

Valores menores indicam posições mais próximas da equipe.

16.2. Proximidade do obstáculo

$$C_{\text{obstaculo}}(x) = \exp \left(-\frac{\|x - p_o\|}{\sigma} \right)$$

Em que:

- p_o : posição observada do obstáculo.
- σ : parâmetro de influência do obstáculo.

Posições próximas do obstáculo recebem maior penalidade.

16.3. Distância de deslocamento

$$C_{\text{deslocamento}}(x) = \min \left(\frac{\|x - p_d\|}{d_{\text{max}}}, 1 \right)$$

Em que:

- p_d : posição atual do drone.
- d_{max} : deslocamento máximo permitido.

Esse termo representa custo geométrico de deslocamento.

Ele não representa consumo energético em joules.

16.4. Conectividade com outros drones

Sempre que o drone já possuir vizinhos conhecidos, a posição candidata deve evitar seu isolamento completo do restante da rede.

Essa condição pode ser representada como restrição:

$$N_{\text{vizinhos previstos}}(x) \geq 1$$

A estimativa deve utilizar informações disponíveis ao drone:

- Últimas posições anunciadas pelos vizinhos.
- Alcance configurado ou estimado da comunicação.
- Atualizações `DroneStatus`.

Se o drone já estiver isolado antes do reposicionamento, essa restrição não deve impedir a busca por uma posição com possibilidade de comunicação.

A conectividade prevista é uma aproximação. A confirmação real ocorre pela transmissão posterior do alerta.

17. Restrições para posições candidatas

Uma posição candidata deve respeitar:

$$0 \leq x \leq 1000$$

$$0 \leq y \leq 1000$$

$$z_{\min} \leq z \leq 20$$

Também deve satisfazer:

- Distância máxima de reposicionamento.
- Distância mínima de segurança em relação ao obstáculo.
- Tempo restante de simulação.
- Condições mínimas de conectividade, quando aplicáveis.
- Coerência com o modelo de mobilidade.

Parâmetros configuráveis:

```
minimumFlightAltitude  
maximumFlightAltitude  
maximumRepositionDistance  
minimumObstacleClearance  
maximumFlightTime
```

O tempo máximo de voo:

$$T_{\text{voo}} = 900 \text{ s}$$

representa apenas um limite operacional.

Não deve ser apresentado como modelo de bateria.

18. Execução do reposicionamento

Após encontrar uma posição viável:

1. O BA entrega a posição ao módulo de mobilidade.
2. O drone inicia o deslocamento.
3. O movimento ocorre gradualmente.
4. O drone permanece integrado à simulação de rede.

5. Ao chegar, o drone registra a distância percorrida.

6. O alerta pendente é retransmitido.

7. A mobilidade Gauss–Markov é retomada.

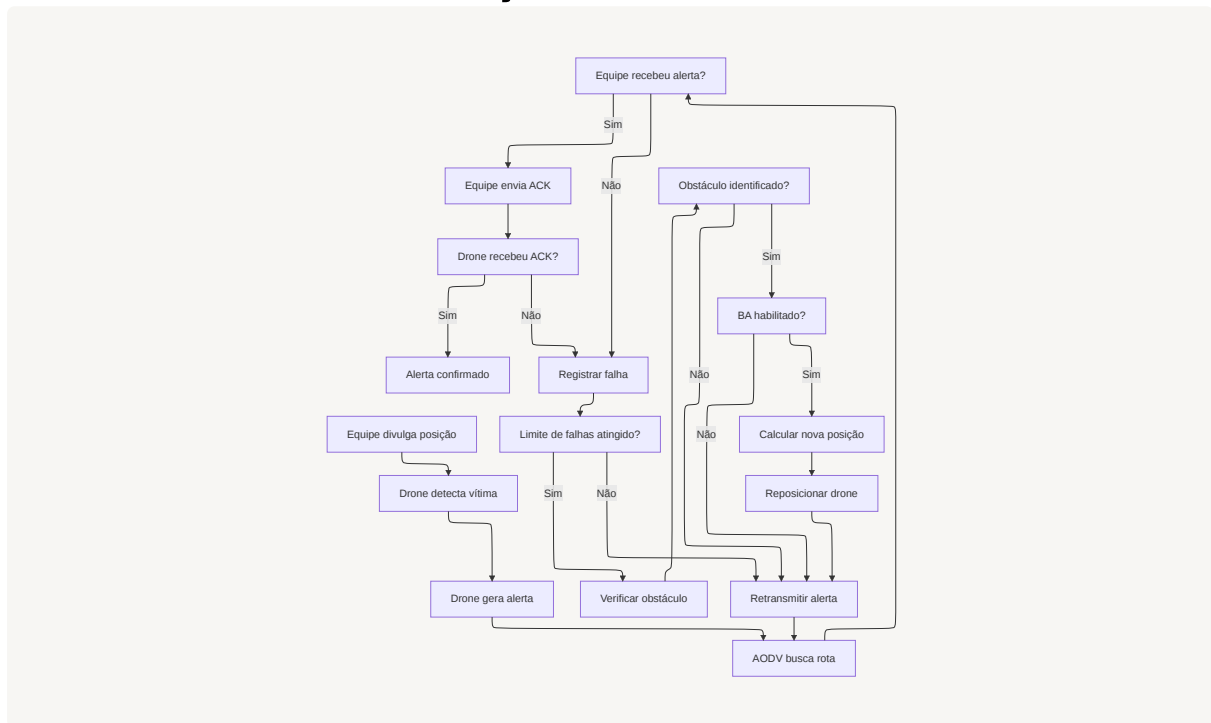
A velocidade utilizada deve respeitar os parâmetros configurados.

O drone não deve:

- Ser teletransportado.
- Iniciar múltiplos reposicionamentos simultâneos.
- Executar movimentos sem limite de distância.
- Utilizar posições fora da área.
- Assumir que o reposicionamento garante entrega.

Cada alerta pode acionar, inicialmente, no máximo um reposicionamento.

19. Fluxo da comunicação



20. Identificadores das mensagens

Cada vítima deve possuir:

victimId

Cada alerta periódico deve possuir:

```
alertId
```

Cada tentativa individual deve possuir:

```
messageId
```

Exemplo:

```
Vítima: V1

Alerta periódico: A1
Tentativa inicial: M1
Primeira retransmissão: M2
Segunda retransmissão: M3

Novo alerta periódico: A2
Tentativa inicial: M4
```

Regras:

- `victimId` permanece igual para a mesma vítima.
- Cada alerta periódico recebe um novo `alertId`.
- Cada retransmissão recebe um novo `messageId`.
- Retransmissões não criam novos alertas lógicos.
- Recebimentos duplicados não aumentam o número de alertas entregues.

21. Entrega e confirmação

A entrega ocorre quando uma equipe recebe um alerta.

A confirmação ocorre quando o drone recebe um ACK válido.

Esses eventos não são equivalentes.

Exemplo:

```
Alerta recebido pela equipe.
ACK perdido durante o retorno.
```

Resultado:

- Alerta entregue.
- Alerta não confirmado.

Um ACK é considerado válido quando corresponde a:

```
alertId
victimId
sourceDroneId
messageId
teamId
```

O endereço da equipe também deve ser coerente com a tentativa original.

22. Métricas de desempenho

A unidade experimental é uma execução completa da simulação.

Cada execução deve registrar:

- G : alertas únicos gerados.
- D : alertas únicos entregues.
- C : alertas únicos confirmados.
- A : tentativas únicas transmitidas.
- U : tentativas únicas recebidas pela equipe.
- R : retransmissões realizadas.
- H : total de saltos dos alertas entregues.

22.1. Taxa de entrega de alertas

$$PDR_{\text{alertas}} = \frac{D}{G}$$

Representa a proporção de alertas lógicos que chegaram a uma equipe.

22.2. Taxa de perda de alertas

$$LossRate_{\text{alertas}} = 1 - PDR_{\text{alertas}}$$

Representa a proporção de alertas que não chegaram a uma equipe.

22.3. Taxa de entrega das tentativas

--

$$PDR_{tentativas} = \frac{U}{A}$$

Representa a proporção de tentativas individuais recebidas pela equipe.

22.4. Taxa de perda das tentativas

$$LossRate_{tentativas} = 1 - PDR_{tentativas}$$

Essa métrica complementa a taxa de perda de alertas.

Um alerta pode ser entregue após várias tentativas perdidas.

22.5. Taxa de confirmação

$$ConfirmationRate = \frac{C}{G}$$

Representa a proporção de alertas cujo recebimento foi confirmado ao drone.

22.6. Retransmissões por alerta

$$RetriesPerAlert = \frac{R}{G}$$

22.7. Atraso médio de entrega

$$MeanDeliveryDelay = \frac{1}{D} \sum_{a=1}^D (t_{entrega,a} - t_{geracao,a})$$

O atraso deve ser calculado apenas para alertas efetivamente entregues.

22.8. Atraso médio de confirmação

$$MeanConfirmationDelay = \frac{1}{C} \sum_{a=1}^C (t_{confirmacao,a} - t_{geracao,a})$$

22.9. Número médio de saltos

$$MeanHopCount = \frac{H}{D}$$

Essa métrica demonstra se os demais drones efetivamente participaram da entrega.

22.10. Entregas com múltiplos saltos

$$MultiHopDeliveryRate = \frac{D_{\text{multissalto}}}{D}$$

Em que:

$$D_{\text{multissalto}}$$

representa o número de alertas entregues por rotas com mais de um salto.

22.11. Métricas do reposicionamento

Registrar:

```
repositionTriggers  
obstaclesDetected  
baActivations  
repositionsStarted  
repositionsCompleted  
repositionDistanceSum  
repositionDurationSum
```

Métricas derivadas:

$$MeanRepositionDistance = \frac{repositionDistanceSum}{repositionsCompleted}$$

$$MeanRepositionDuration = \frac{repositionDurationSum}{repositionsCompleted}$$

Essas métricas representam custo operacional de movimento.

Não devem ser apresentadas como consumo energético.

23. Regras de consistência das métricas

As seguintes condições devem ser respeitadas:

$$D \leq G$$

$$C \leq D$$

$$U \leq A$$

$$repositionsCompleted \leq repositionsStarted$$

$$repositionsStarted \leq baActivations$$

$$obstaclesDetected \leq repositionTriggers$$

Se o denominador de uma razão for zero, o resultado deve ser tratado como indefinido.

Não substituir automaticamente valores indefinidos por zero.

24. Registro dos resultados no OMNeT++

As métricas devem ser registradas preferencialmente por:

- Signals.
- Scalars.
- Vectors.
- Estatísticas declaradas nos módulos NED.

Arquivos esperados:

```
.sca
.vec
```

Os contadores finais devem ser registrados como escalares.

Exemplos:

```
generatedAlerts
deliveredAlerts
confirmedAlerts
transmissionAttempts
receivedAttempts
applicationRetries
multiHopDeliveries
baActivations
repositionsCompleted
```

Os atrasos e reposicionamentos podem ser registrados como vetores quando necessário:

```
deliveryDelay
confirmationDelay
hopCount
```

```
repositionDistance  
repositionDuration
```

Os visualizadores do INET podem auxiliar na inspeção do cenário, mas não substituem o registro de métricas.

Arquivos PCAP podem ser utilizados para auditoria pontual.

Não é necessário manter PCAP habilitado em todas as execuções principais se isso aumentar excessivamente o volume de dados.

A análise posterior pode ser realizada com Python.

25. Configurações experimentais

25.1. Cenário com uma vítima

Elemento	Configuração
Drones	4
Vítimas	1
Drones gerando alertas	1
Equipes	1, 5, 10 ou 15
Obstáculos	2
Tempo	900 s
Repetições	30
Comparação	BA desligado versus BA ligado

25.2. Cenário com duas vítimas

Elemento	Configuração
Drones	4
Vítimas	2
Drones gerando alertas	2
Equipes	1, 5, 10 ou 15
Obstáculos	2
Tempo	900 s
Repetições	30
Comparação	BA desligado versus BA ligado

25.3. Quantidade de execuções

Considerando:

- 2 configurações de vítimas.
- 4 configurações de equipes.
- 2 condições do BA.
- 30 repetições.

O total é:

$$N_{\text{execucoes}} = 2 \times 4 \times 2 \times 30$$

Portanto:

$$N_{\text{execucoes}} = 480$$

Cenários sem obstáculos podem ser executados separadamente como referência para verificar o efeito da obstrução.

26. Pareamento das execuções

As condições com BA ligado e desligado devem utilizar:

- Mesma seed.
- Mesma posição inicial.
- Mesmos obstáculos.
- Mesmas vítimas.
- Mesmos parâmetros de rede.
- Mesmos eventos iniciais.
- Mesmas regras de mobilidade.

A diferença experimental deve ser:

```
baEnabled = false
```

ou:

```
baEnabled = true
```

Os fluxos aleatórios relacionados à mobilidade, geração de mensagens e funcionamento do algoritmo devem ser organizados para evitar interferência indevida entre cenários.

O reposicionamento naturalmente altera a trajetória posterior do drone. Essa alteração faz parte do tratamento avaliado.

A unidade de comparação é:

$$\Delta_i = Y_{\text{BA ligado},i} - Y_{\text{BA desligado},i}$$

Em que:

- i : identificação da seed.
- Y : métrica avaliada.

27. Análise estatística

Para cada configuração, apresentar:

- Média.
- Mediana, quando pertinente.
- Desvio padrão.
- Intervalo de confiança.
- Quantidade de repetições.
- Diferença entre BA ligado e desligado.

A análise estatística deve considerar o pareamento das seeds.

O teste escolhido deve ser compatível com a métrica:

- Teste pareado para diferenças aproximadamente contínuas.
- Teste não paramétrico ou de permutação quando necessário.
- Análise específica para resultados binários quando houver poucos eventos por execução.

Não utilizar pacotes individuais como se fossem repetições independentes.

A repetição estatística é a execução completa.

28. Testes obrigatórios

28.1. Teste de comunicação direta

Verificar:

```
Drone envia alerta.  
Equipe recebe alerta.  
Equipe envia ACK.  
Drone recebe ACK.
```

28.2. Teste de comunicação multissalto

Verificar:

```
Drone emissor envia alerta.  
Drone intermediário encaminha.  
Equipe recebe.  
ACK retorna ao drone emissor.
```

Esse teste confirma que a FANET não está limitada a enlaces diretos.

28.3. Teste de obstáculo

Verificar:

```
Enlace sem obstrução.  
Inclusão do obstáculo.  
Atenuação ou perda do pacote.
```

28.4. Teste do sensor

Verificar:

```
Obstáculo na direção observável.  
Sensor identifica obstáculo.  
Posição do obstáculo é retornada.
```

Também verificar:

```
Obstáculo fora do alcance.  
Sensor não identifica obstáculo.
```

28.5. Teste do reposicionamento

Verificar:

```
Falha de confirmação.  
Identificação do obstáculo.  
Execução do BA.  
Escolha de posição viável.  
Movimento gradual.  
Nova tentativa de transmissão.
```

28.6. Teste de preservação da conectividade

Verificar:

```
Drone possui vizinho antes do movimento.  
Nova posição não isola desnecessariamente o drone.  
Encaminhamento pela rede continua disponível.
```

28.7. Teste de deduplicação

Verificar:

```
Mesmo alertId.  
Diferentes messageId.  
Uma única entrega lógica.
```

28.8. Teste das métricas

Verificar:

```
Alertas gerados.  
Alertas entregues.  
Alertas confirmados.  
Tentativas transmitidas.  
Tentativas recebidas.  
Retransmissões.  
Atrasos.  
Saltos.  
Reposicionamentos.
```

28.9. Teste de comparação experimental

Verificar:

```
Mesma seed.  
Mesmo cenário.  
Mesmo número de vítimas.  
Mesmo número de equipes.
```

BA desligado.
BA ligado.

29. Limitações que devem ser declaradas

A implementação considera:

- Obstáculos estáticos.
- Identificação abstrata das vítimas.
- Sensor geométrico simplificado.
- Informações de posição potencialmente desatualizadas.
- Estimativa geométrica para escolher a nova posição.
- Ausência de modelo energético detalhado.
- Ausência de processamento de imagens.
- Utilização do AODV sem modificações.
- Resultados dependentes dos cenários e parâmetros configurados.

Não afirmar que:

- O sistema comprova resiliência sem uma métrica específica.
- O BA sempre encontra uma posição adequada.
- Toda perda é provocada por obstáculo.
- A distância de deslocamento equivale a consumo energético.
- O tempo de voo representa carga real de bateria.
- Uma confirmação perdida significa necessariamente que o alerta não chegou.
- Obstáculos fixos são dinâmicos.
- O projeto altera o AODV.
- Um cenário exclusivamente direto avalia conectividade multissalto.

30. Síntese da implementação

O projeto deve implementar uma FANET no OMNeT++ com INET, composta por drones móveis, equipes de resgate, vítimas fixas e obstáculos estáticos. As equipes divulgam suas posições, e os drones responsáveis pelas vítimas enviam alertas periódicos utilizando UDP e o AODV padrão, com possibilidade de encaminhamento por outros drones. Quando um alerta deixa de receber

confirmação após um número configurado de tentativas, o drone verifica a presença de um obstáculo por meio de um sensor geométrico abstrato. Caso a política esteja habilitada, o Algoritmo do Morcego seleciona uma posição considerando a equipe, o obstáculo, o deslocamento necessário e a conectividade com drones vizinhos. Após o reposicionamento, o alerta é retransmitido. A avaliação compara cenários com e sem reposicionamento utilizando métricas de entrega, perda, confirmação, atraso, retransmissões, múltiplos saltos e movimentação.